Computing Lyapunov exponents of chaotic systems

Joe Dearing

Student Number: 31016744

Department of Mathematics and Statistics

University of Reading

May 2025

Introduction

Lyapunov exponents help describe the behaviour of dynamical systems. They are a measure of how quickly an infinitesimally small distance between two initial points evolves with time, which in turn helps quantify how chaotic a system is. Throughout this report we will calculate Lyapunov exponents through numerous methods for various systems and observe the behaviour of such systems.

Declaration

I certify that this is my own work and that use of material from other sources has been properly and fully acknowledged in the text. I have read the University's definition of plagiarism and the department's advice on good academic practice. I understand that the consequence of committing plagiarism, if proven and in the absence of mitigating circumstances may include failure in the Year or Part or removal from the membership of the University. I also certify that neither this piece of work, nor any part of it, has been submitted in connection with another assessment.

In submitting this work, I can confirm that I have not used Generative AI Tools to prepare and complete my assignment.

1. Computing Lyapunov exponents by hand

For a one-dimensional discrete-time map

$$x_{i+1} = f(x_i)$$

Where i is the time index, the Lyapunov exponent given an initial condition x_0 is

$$\lambda(x_0) = lim_{n\to\infty}\frac{1}{n}\sum_{i=0}^{n-1}\ln|f'(x_i)|$$

Given the time map

$$x_{i+1} = f(x_i) = \begin{cases} 2x_i \; for \; x_i < \frac{1}{2} \\ 2(1-x_i) \; for \; x_i \geq \frac{1}{2} \end{cases}$$

With x_0 between 0 and 1 we find that

$$f'(x_i) = \begin{cases} 2 \; for \; x_i < \frac{1}{2} \\ -2 \; for \; x_i \geq \frac{1}{2} \end{cases} \Longrightarrow |f'(x_i)| = 2 \; \forall x_i : i \; \in \{0\} \cup \mathbb{N}$$

Therefore

$$\lambda(x_0) = lim_{n\to\infty}\frac{1}{n}\sum_{i=0}^{n-1}\ln|f'(x_i)| = lim_{n\to\infty}\frac{1}{n}\sum_{i=0}^{n-1}\ln 2 = \ln 2 \; lim_{n\to\infty}\frac{1}{n}\sum_{i=0}^{n-1}1 = \ln 2 \; lim_{n\to\infty}\frac{n}{n}$$

$$lim_{n\to\infty}\frac{n}{n} = lim_{n\to\infty}1 = 1$$

Hence the Lyapunov exponent for this map, given x_0 is between 0 and 1, is $\lambda(x_0) = \ln 2$.

2. Using computer programs to approximate Lyapunov exponents

Here we consider the following time map.

$$x_{i+1} = rx_i(1-x_i)$$

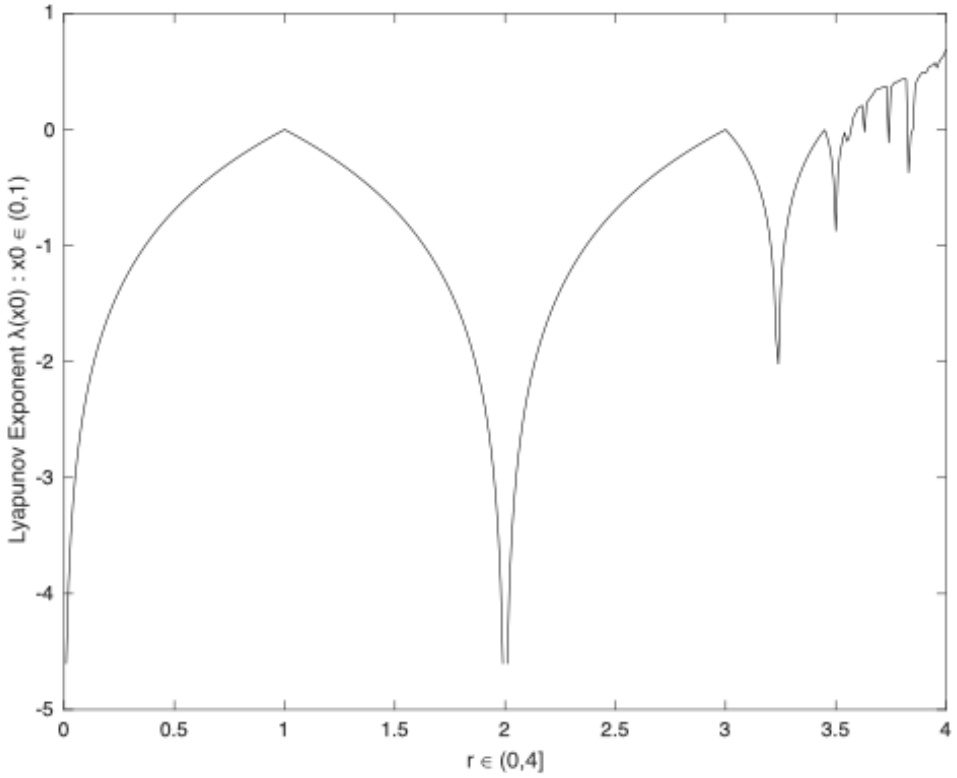
Where r is a parameter.

2a)

For $r = 3.5$ and x_0 a random number between 0 and 1 we find that, using Code 1, the Lyapunov exponent for such system is $\lambda(x_0) = -0.8725 \; to \; 4 \; d.p.$ for all such x_0.

2b)

The following graph, produced using Code 2, shows how the Lyapunov exponent varies compared to the parameter r. x_0 is again set to be a random number between 0 and 1. The output of said code remains unchanged for all such x_0.

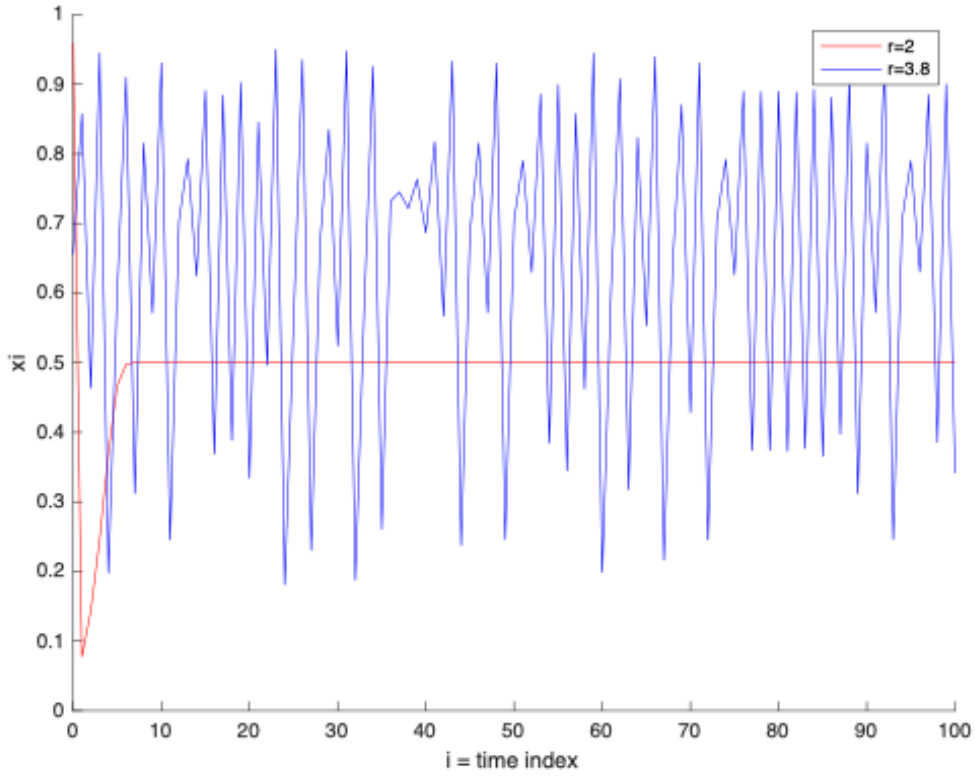


To note, the Lyapunov exponent of the time map when $r = 2$, tends to negative infinity. And that the Lyapunov exponent is undefined when $r = 0$, since you can't evaluate the logarithm of zero.

2c)

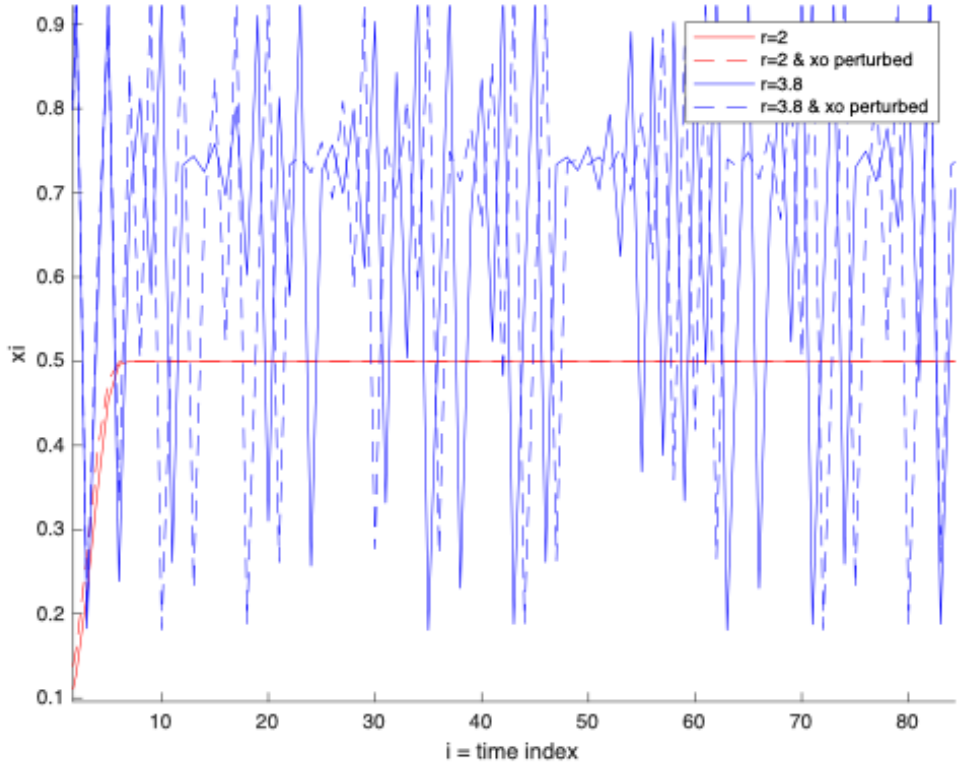
Firstly, we must note that having a positive Lyapunov exponent is necessary for chaos. The following graph, produced using Code 3, shows how the time-series of length 100 differs for $r = 2$ and $r = 3.8$. When $r = 2$, the time-series has a Lyapunov exponent that tends to negative infinity, hence we expect the time-series for such to behave less erratically than that of when $r = 3.8$, since the Lyapunov exponent of such time-series is positive.

x_0 is again chosen to be a random number between 0 and 1. For the plot below $x_0 = 0.9595\ and\ x_0 = 0.6557\ to\ 4\ d.p.$ when $r = 2\ and\ r = 3.8$ respectivly.



When $r = 2$, the time series converges to $x = 0.5$ and it does so in a non-chaotic manner. The behaviour of such system could be easily modelled using various methods. When $r = 3.8$, the time-series behaves erratically, not appearing to have any sense of convergence. In this case it would be difficult to produce a model to copy such behaviour due to its chaotic nature.

2d)



For each of the time series with the previous parameter values, we now perturb x_0 by a small amount, in this case by $\delta = 0.01$. Plotting each time series with x_0 and $x_0 + \delta$ yields the graph above, produced using Code 4. In the following $x_0 = 0.0357\ and\ x_0 = 0.8491\ to\ 4\ d.p.$ when $r = 2\ and\ r = 3.8$ respectivly.

For the non-chaotic system, we can see that the time-series follows the same trends, being that it converges to 2 in a similar timeframe. Furthermore, we can see that it does so in a similar fashion as, in the graph above, we can see that both time series are monotonic increasing functions when $r = 2$.

For the chaotic system, initially the time-series appear to follow the same trend, but they quickly diverge from one another, following different trends and patterns. This implies that even with a small change in initial data, the time-series will follow pathways that vary significantly from one another. Hence resulting in an unpredictable and chaotic system.

3. Lyapunov exponents of systems of ordinary differential equations

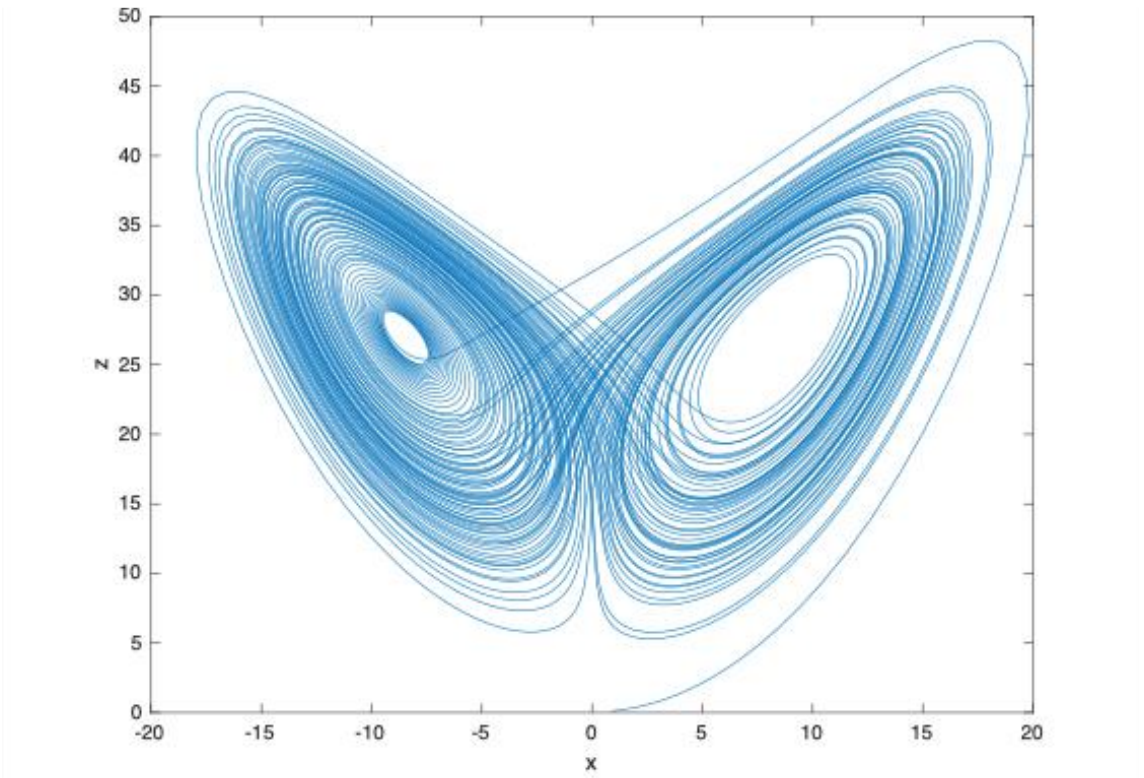
The following system has d Lyapunov exponents, collectively known as the Lyapunov spectrum.

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{g}(\boldsymbol{x}) : \boldsymbol{x} \in \mathbb{R}^d$$

3a)

Here we consider the following system, a specific version of the Lorenz system.

$$\frac{dx}{dt} = 10(y - x), \qquad \frac{dy}{dt} = x(28 - z) - y, \qquad \frac{dz}{dt} = xy - \frac{8}{3}z$$



Solving this system numerically for $t \in [0,100]$ using the 4th order Runge-Kutta method with a timestep of 0.01 yields the above solution in the x-z plane, produced using Code 5. The initial condition for the x, y and z coordinates were again a random number between 0 and 1.

3b)

Using the algorithm in (Sprott, 2015), the maximum Lyapunov exponent of this system is estimated to be between $0.90\ and\ 0.91\ to\ 2\ d.p.$ Some examples of the output of the code are given below.

```
>> PPRCHAOS6
    0.9074

>> PPRCHAOS6
    0.9046

>> PPRCHAOS6
    0.9067
```

This result was achieved through Code 6 and the iteration function. To note this range of results occurred regardless of the initial conditions of the coordinates and the length of d0. I predict that the variation in results is due to not repeating step 4 to 6 in (Sprott, 2015) enough. However, due to limited computing resources available I was not able to repeat such steps further.

3c)

We know wish to consider, as outlined in (Sandri, 1996), how to calculate the full Lyapunov spectrum of the dynamical system

$$\dot{x} = F(x) : \dot{x} = \frac{dx}{dt}, x = x(t) \in \mathbb{R}^n\ and\ F: U \to \mathbb{R}^n$$

where U is an open set in the phase space $\mathbb{R}^n$ and F has at least continuous derivatives of order 1. This can be easily translated into a system of ordinary differential equations. We say that F generates the flow f where $f^t(x) = f(x,t)$, which again in turn has at least continuous derivatives of order 1, such that

$$f: U \times \mathbb{R} \to \mathbb{R}^n,\ \dot{f}^t(x) = F\big(f^t(x)\big)\ \forall x \in U, t \in \mathbb{R}$$

Given an initial condition $x_0 \in U$ the solution to our system is then the set known as the trajectory of the system through x_0

$$\{f^t(x_0): t \in \mathbb{R}\} : f^0(x_0) = x_0$$

Finally, we need to define a limit set and the basin of attraction. A point p is a ω-limit point of x if there are points on the trajectory of x such that $f^t(x) \to p\ as\ t \to \infty$. $\Omega(x)$ is the set of all such points of x. The limit set Ω is attracting if there exists an open set $U \subset \Omega(x)$ such that $\Omega(x) = \Omega\ \forall x \in U$. The basin of attraction of Ω is the union of all such open sets U.

Consider two points x_0 and $x_0 + u_0$ where u_0 is a small perturbation. After time t this perturbation becomes

$$u_t = f^t(x_0 + \delta_0) - f^t(x_0) = D_{x_0} f^t(x_0) \cdot u_0$$

Where the last term is obtaining by linearizing f^t, D_{x_0} denoting differentiation by x_0 which since f is vector valued is Jacobian. Then we can define the average exponential rate of change of the two trajectories of such points by

$$\lambda(x_0, u_0) = lim_{t\to\infty} \frac{1}{t} ln \frac{\|u_t\|}{\|u_0\|} = lim_{t\to\infty} \frac{1}{t} ln \left\| D_{x_0} f^t(x_0) \cdot u_0 \right\|$$

It can be shown that under weak conditions this limit exists and is equal to the largest Lyapunov exponent for almost all x_0 and δ_0. Throughout this report we have only considered Lyapunov exponents of order 1, we now wish to define such of order p: $1 \leq p \leq n$

$$\lambda^p(x_0, U_0) = lim_{t\to\infty} \frac{1}{t} \ln[Vol^p(D_{x_0} f^t(U_0))]$$

Where we consider a parallelepiped U_0 in the tangent space defined by the p vectors $u_i : i = 1, \dots, p$. Vol^p being the p-dimensional volume defined in such space. These Lyapunov exponents describe the rate of change of a p-dimensional volume in such space.

A theorem in (Oseledec, 1968) shows that you can find p linearly independent vectors u_i such that

$$\lambda^p(x_0, U_0) = \sum_{i=1}^{p} \lambda_p$$

Where U_0 is again parallelepiped by such linearly independent vectors.

We now consider our system with an initial point lying in the basin of attraction of the limit set. It can be shown that u_t evolves in time satisfying the equation

$$\dot{\Phi}_t(x_o) = D_x F\big(f^t(x_0)\big) \cdot \Phi_t(x_0),\ \Phi_0(x_0) = I$$

Where $\Phi_t(x_0)$ is the derivative with respect to x_0 of $f^t(x_0)$. This is a matrix-valued differential equation that depends on our system $\dot{x} = F(x)$. So, to calculate the trajectory we must integrate the combined system

$$\dot{x} = F(x), x(t_0) = x_0$$

$$\dot{\Phi} = D_x F(x) \cdot \Phi, \Phi(t_0) = I$$

To note the initial condition is simply the identity matrix of corresponding size.

We now wish to find a suitable method for determining a set of linearly independent vectors that satisfy the above theorem from (Oseledec, 1968). To do this we now apply the following algorithm which is described in (Benettin, 1980). Firstly, given a set of p linearly independent vectors u_i in $\mathbb{R}^n$ through the Gram-Schmidt procedure we can generate an orthonormal set of vectors v_i that span the same subspace. These vectors are given by

$$w_1 = u_1, \qquad v_1 = w_1/\|w_1\|$$

$$w_2 = u_2 - \langle u_2, v_1\rangle v_1, \qquad v_2 = w_2/\|w_2\|$$

$$w_p = u_p - \sum_{i=1}^{p-1} \langle u_p, v_i\rangle\, v_i, \qquad v_p = w_p/\|w_p\|$$

It can be shown that volume of the parallelepiped spanned by the vectors u_i is

$$Vol\{u_1, \dots, u_p\} = \|w_1\| \dots \|w_p\|$$

We start the algorithm by initialising x_0 and $U_0 = [u_1^0, \dots, u_p^0]$ an $n \times n$ matrix. The vectors u_i in such matrix can be any set of p linearly independent vectors, with dimensions corresponding to our system. Using the Gram-Schmidt procedure we generate the corresponding matrix of orthonormal vectors $V_0 = [v_1^0, \dots, v_p^0]$. Where the vectors v_i are calculated using the corresponding vectors u_i

from U_o. Then we integrate the combined system above from $\{x_0, V_0\}$ over a short interval T, to obtain

$$x_1 = f^T(x_0)\ and\ U_1 = \left[u_1^1, \dots, u_p^1\right] = D_{x_0} f^T(U_0) = \Phi_t(x_0) \cdot [u_1^0, \dots, u_p^0]$$

Again, we orthonormalize U_1 to obtain V_1, allowing us to again integrate the combined system from $\{x_1, V_1\}$ to obtain x_2 and U_2. We continue to repeat such procedure K times. During the k-th step, the p-dimensional volume as defined in the equation for λ^p increases by a factor of $\|w_1^k\| \dots \|w_p^k\|$ where the w_i^k's are the orthogonal vectors calculated during the Gram-Schmidt procedure with U_k. This implies that

$$\lambda^p(x_0, U_0) = lim_{k\to\infty} \frac{1}{kT} \sum_{i=1}^{k} \ln(\|w_1^i\| \dots \|w_p^i\|)$$

Hence by subtracting λ^{p-1} from λ^p, and the fact that using these vectors from the above method meet the requirements for the theorem in (Oseledec, 1968) to hold, we find that

$$\lambda_p = lim_{k\to\infty} \frac{1}{kT} \sum_{i=1}^{k} \ln\|w_p^i\|$$

Where λ_p is the p-th Lyapunov exponent of the dynamical system such that $1 \leq p \leq n$ and $p \in \mathbb{N}$. Hence, we can calculate the full Lyapunov spectrum of such system.

Throughout this paper we have mainly calculated the maximum Lyapunov exponent for systems of dimensions ranging from one to three, labelling it as a necessary condition for chaos. In a multidimensional system this exponent corresponds to dimension that exhibits the greatest rate of divergence between two initially close states. The Lyapunov spectrum consists of p Lyapunov exponents for a p-dimensional system. Each of these Lyapunov exponents λ_i quantify the average exponential rate of expansion or contraction in the i^{th} dimension or axis. In general, a positive Lyapunov exponent implies expansion and a negative Lyapunov exponent implies contraction in the corresponding axes which can help us identify if a system is chaotic or not.

Bibliography

References

Sprott, J. C. (2015). Numerical Calculation of Largest Lyapunov Exponent

https://sprott.physics.wisc.edu/chaos/lyapexp.htm

Sandri, M. (1996). Numerical Calculation of Lyapunov Exponents, The Mathematica Journal, Miller Freeman

Oseledec, V. I. (1968) A multiplicative ergodic theorem: Lyapunov characteristic numbers for dynamical systems, Trans. Moscow Math. Soc. 19:197-231

Benettin, G. (1980) Lyapunov characteristic exponents for smooth dynamical systems and for Hamiltonian systems: A method for computing all of them. Part 2: Numerical application, Meccanica 15:21-30

Coding Files – All programmed using MatLab

Code 1 – PPRCHAOS, used in question 2a

```
x=rand;
r=3.5;
sum=0;
for n = 1:1000000 %number of iterations found to be sufficient for accuracy to 4.d.p.
   sum=sum+log(abs(r*(1-2*x)));
   x=r*x*(1-x); %loop structured so sum calculated for i = 0 to n-1
end
lambda=sum/1000000;
disp(lambda)
```

Code 2 – PPRCHAOS2, used in question 2b

```
x=rand;
r=linspace(0,4,401);
xvector(1:401)=x;
sum=0*r;
for n = 1:1000000 %number of iterations found to be sufficient for accuracy to 4.d.p.
   sum=sum+log(abs(r.*(1-2*xvector)));
   xvector=r.*(xvector.*(1-xvector)); %loop structured so sum calculated for i = 0 to n-1
end
lambda=sum/1000000;
plot(r,lambda,Color='black')
```

```
xlabel('r ∈ (0,4]')
ylabel('Lyapunov Exponent λ(x0) : x0 ∈ (0,1)')
```

Code 3 – PPRCHAOS3, used in question 2c

```
xvector1=zeros(1,101);
xvector1(1)=rand;
r1=2;

xvector2=zeros(1,101);
xvector2(1)=rand;
r2=3.8;

for i = 1:100
    xvector1(i+1)=r1*xvector1(i)*(1-xvector1(i));
    xvector2(i+1)=r2*xvector2(i)*(1-xvector2(i));
end

timeindex=linspace(0,100,101);

hold on
plot(timeindex,xvector1,Color='red')
plot(timeindex,xvector2,Color='blue')
xlim([0,100])
xlabel('i = time index')
ylabel('xi')
legend('r=2','r=3.8')
```

Code 4 – PPRCHAOS4, used in question 2d

```
delta=0.01;

xvector1=zeros(1,101);
xvector1(1)=rand;
xvector1p=zeros(1,101);
xvector1p(1)=xvector1(1)+delta;
r1=2;
```

```
xvector2=zeros(1,101);
xvector2(1)=rand;
xvector2p=zeros(1,101);
xvector2p(1)=xvector2(1)+delta;
r2=3.8;
for i = 1:100
    xvector1(i+1)=r1*xvector1(i)*(1-xvector1(i));
    xvector1p(i+1)=r1*xvector1p(i)*(1-xvector1p(i));
    xvector2(i+1)=r2*xvector2(i)*(1-xvector2(i));
    xvector2p(i+1)=r2*xvector2p(i)*(1-xvector2p(i));
end

timeindex=linspace(0,100,101);

hold on
plot(timeindex,xvector1,Color='red')
plot(timeindex,xvector1p,Color='red',LineStyle='--')
plot(timeindex,xvector2,Color='blue')
plot(timeindex,xvector2p,Color='blue',LineStyle='--')
xlim([0,100])
xlabel('i = time index')
ylabel('xi')
legend('r=2','r=2 & xo perturbed','r=3.8','r=3.8 & xo perturbed')
```

Code 5 – PPPRCHAOS5, used in question 3a

```
h=0.01; %timestep

xv=zeros(1,10001);
yv=zeros(1,10001);
zv=zeros(1,10001);

xv(1)=rand;
yv(1)=rand;
zv(1)=rand;
```

```
f=@(x,y)(10*(y-x));
g=@(x,y,z)(x*(28-z)-y);
j=@(x,y,z)(x*y-(8*z)/3);

for i = 1:10000 %RK4 for t element of [0,100]
    k1=f(xv(i),yv(i));
    l1=g(xv(i),yv(i),zv(i));
    m1=j(xv(i),yv(i),zv(i));

    k2=f(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1);
    l2=g(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1,zv(i)+0.5*h*m1);
    m2=j(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1,zv(i)+0.5*h*m1);

    k3=f(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2);
    l3=g(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2,zv(i)+0.5*h*m2);
    m3=j(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2,zv(i)+0.5*h*m2);

    k4=f(xv(i)+h*k3,yv(i)+h*l3);
    l4=g(xv(i)+h*k3,yv(i)+h*l3,zv(i)+h*m3);
    m4=j(xv(i)+h*k3,yv(i)+h*l3,zv(i)+h*m3);

    xv(i+1)=xv(i)+(h/6)*(k1+2*k2+2*k3+k4);
    yv(i+1)=yv(i)+(h/6)*(l1+2*l2+2*l3+l4);
    zv(i+1)=zv(i)+(h/6)*(m1+2*m2+2*m3+m4);

end

plot(xv,zv)
xlabel('x')
ylabel('z')
```

Code 6 – PPRCHAOS6, used in question 3b

```
x0=[rand,rand,rand]; %step 1

[x,y,z]=iteration(x0(1),x0(2),x0(3),10000); %step 2, using iteration function
```

```
xa=[x,y,z];

d0=10^-8;
xb=[xa(1)+d0,xa(2),xa(3)]; %step 3, choosing to perturb in x-axis

p=zeros(1,10000); %vector containing |d1/d0| values
for i = 1:10000 %step, 4-6 repeated

   [x1,y1,z1]=iteration(xa(1),xa(2),xa(3),100);
   xa1=[x1,y1,z1];
   [x2,y2,z2]=iteration(xb(1),xb(2),xb(3),100);
   xb1=[x2,y2,z2];
   d1=norm(xa1-xb1);

   p(i)=log(abs(d1/d0));

   xa=xa1;
   xb=[xa1(1)+d0*(xb1(1)-xa1(1))/d1,xa1(2)+d0*(xb1(2)-xa1(2))/d1,xa1(3)+d0*(xb1(3)-xa1(3))/d1];

end

disp(mean(p)) %estimate of largest lyapunov exponent
```

- Iteration – function used in PPRCHAOS6.

```
function [xnew,ynew,znew] = iteration(x0,y0,z0,n) %code from PPRCHAOS5, altered to be in form of
function
h=0.01;

xv=zeros(1,n+1);
yv=zeros(1,n+1);
zv=zeros(1,n+1);

xv(1)=x0;
yv(1)=y0;
zv(1)=z0;
```

```
f=@(x,y)(10*(y-x));
g=@(x,y,z)(x*(28-z)-y);
j=@(x,y,z)(x*y-(8*z)/3);

for i = 1:n
    k1=f(xv(i),yv(i));
    l1=g(xv(i),yv(i),zv(i));
    m1=j(xv(i),yv(i),zv(i));

    k2=f(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1);
    l2=g(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1,zv(i)+0.5*h*m1);
    m2=j(xv(i)+0.5*h*k1,yv(i)+0.5*h*l1,zv(i)+0.5*h*m1);

    k3=f(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2);
    l3=g(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2,zv(i)+0.5*h*m2);
    m3=j(xv(i)+0.5*h*k2,yv(i)+0.5*h*l2,zv(i)+0.5*h*m2);

    k4=f(xv(i)+h*k3,yv(i)+h*l3);
    l4=g(xv(i)+h*k3,yv(i)+h*l3,zv(i)+h*m3);
    m4=j(xv(i)+h*k3,yv(i)+h*l3,zv(i)+h*m3);

    xv(i+1)=xv(i)+(h/6)*(k1+2*k2+2*k3+k4);
    yv(i+1)=yv(i)+(h/6)*(l1+2*l2+2*l3+l4);
    zv(i+1)=zv(i)+(h/6)*(m1+2*m2+2*m3+m4);

end

xnew=xv(n+1);
ynew=yv(n+1);
znew=zv(n+1);

end
```